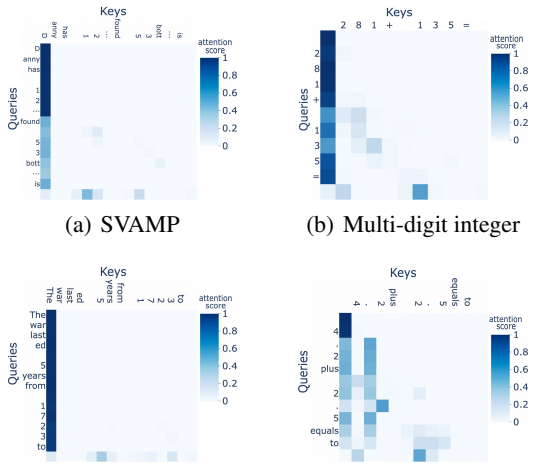


(a) Reception of {A} and {B} (b) Generation of {C} (c) Steps of Calculation

Figure 6: We investigate the projection of each MLP layer input or output along the direction of number token {A}, {B}, and {C}, respectively. The x-axis represents the layer number, ranging from 0 to 31, while the y-axis represents the cosine similarity between the embeddings of the MLP input or output and the number tokens.

ring to other datasets (shown in Figure 4). For more case studies on the key heads, such as the attention pattern on operators, please refer to Figure 16 in Appendix F.



(a) SVAMP (b) Multi-digit integer

(c) Diff. data format (d) Rational numbers

Figure 7: The transferability of attention patterns in key heads on the unseen samples in Figure 4, which mainly attend to the number operands.

Key MLPs behavior. In Figure 6(a), we conduct an initial investigation of the similarities between the MLP_{in} and tokens {A} and {B} over 1000 samples, to verify the information of operands received from above analyzed attention heads. For the 0–12th layers, both $\langle MLP_{in}, \{\texttt{A}\}\rangle$ and $\langle MLP_{in}, \{\texttt{B}\}\rangle$ are close to zero. It indicates no operands are captured during this stage, which corresponds to the blank region (*i.e.*, few key heads for computation task) before the 12th layer in Figure 2. For the 12–17th layers, we observe a sharp increase in the similarities with both operands ({A} and {B}). This surge corresponds to the presence of key attention heads, *e.g.*, 12.22/13.11 in layer 12/13, indicating that the operands are progressively being collected and "written" into the MLPs of these layers for subsequent computations. In layers 17–31, the similarities $\langle MLP_{in}, \{\texttt{A}\}\rangle$ and $\langle MLP_{in}, \{\texttt{B}\}\rangle$ gradually decrease, signifying the transition into a new stage that digests the input information for generating the answers.

To understand how each MLP layer contributes to generating the correct answer {C}, we compute the similarity between token {C} and the input/output of the MLPs. We use $\langle MLP_{out} - MLP_{in}, W_U[\{\texttt{C}\}]\rangle$ to reflect the direct contribution of the MLP to the correct answer, and $\langle MLP_{out} - MLP_{in}, W_U[Other]\rangle$ for other candidate numbers (as shown in Figure 6(b)). Starting from the 17th layer, where the MLPs begin processing operand information, we observe a noticeable increase in $\langle MLP_{out} - MLP_{in}, W_U[\{\texttt{C}\}]\rangle$ and a decrease in $\langle MLP_{out} - MLP_{in}, W_U[Other]\rangle$. This trend indicates that these MLPs are gradually carrying out the calculation required for the correct answer. The above ascending and descending trends can also be viewed in other LLMs as in Figure 13 and Figure 14 in Appendix C.

Based on the above analyses, we further delve into the detailed calculation process from layer 17 to 28. We investigate a case of "$4 + 3 =$" and analyze $MLP_{out} - MLP_{in}$ compared to all numeric tokens in Figure 6(c). At layers 17 and 19, the numbers '3' and '4' are at the top, indicating that MLPs receive and store input {A} ='4' and {B} ='3', respectively. After that, the numbers '6' and '1' appear top at the subsequent layers 20 and 21. In summary, the LLM predicts the next token as '7' in a single inference. However, within the LLM's architecture, the answer '7' is the result of a collaborative process across multiple layers 22/23/25/27, after the layers 17/19/20/21 generate '3'/'4'/'6'/'1', respectively. The results demonstrate that the answer '7' is not deduced directly, and MLPs perform calculations in a "layer-by-layer" manner, somewhat akin to the addition process in computers (a comparison of these two processes are presented Appendix H). Additionally, we observe that numbers close to the correct answer, such as '6' and '8', also appear

Table 1: Overall performance. We evaluate the capabilities of LLaMA2-7B and LLaMA2-13B, transitioning from generic tasks (*e.g.*, MMLU and CSQA) to mathematical tasks (*e.g.*, GSM8K, AddSub, SingleEq, and SVAMP). Supervised fine-tuning across the entire parameter set (denoted as Full SFT) leads to enhanced performance in math-related tasks, albeit at the expense of its capabilities in generic tasks. In contrast, selectively tuning only the parameters of 32 critical attention heads (denoted as Precise SFT) yields comparable improvements while preserving the model's proficiency in generic tasks, with faster training speed (samples processed per second) and less tuned parameters.

			Mathematical Tasks								Generic Tasks			
			GSM8K		AddSub		SingleEq		SVAMP		MMLU		CSQA	
Models	**Train Speed**	**Tuned Params.**	**Acc.**	**Δ**	**Acc.**	**Δ**	**Acc.**	**Δ**	**Acc.**	**Δ**	**Acc.**	**Δ**	**Acc.**	**Δ**
LLaMA2-7B	-	-	14.6	-	30.5	-	65.4	-	34.7	-	46.0	-	59.8	-
+ Full SFT	15sam./sec.	6.7B	24.6	+10.0	53.7	+23.2	68.2	+2.8	50.3	+15.6	40.5	-5.5	54.0	-5.8
+ Precise SFT	50sam./sec.	0.07B	27.4	+12.8	50.6	+20.1	69.7	+4.3	55.8	+21.1	46.4	+0.4	59.6	-0.2
LLaMA2-13B	-	-	28.7	-	33.7	-	76.6	-	45.7	-	54.8	-	67.3	-
+ Full SFT	8sam./sec.	13.0B	44.6	+15.9	62.2	+28.5	79.8	+3.2	62.8	+17.1	50.2	-4.6	62.0	-5.3
+ Precise SFT	34sam./sec.	0.08B	46.3	+17.6	61.1	+27.4	82.2	+5.6	66.6	+20.9	55.0	+0.2	67.2	-0.1

at the top in layer 23. However, in subsequent layers, the correct answer '7' consistently remains top while '6' and '8' decline. It indicates that LLMs may do computations in a coarse-to-fine manner, where the result is firstly regressed to an embedding around that of the right answer, and then converges to the final output based on the fine-grained information introduced by subsequent MLPs.

Consolidating these findings, we can assert with some confidence that LLMs initially leverage attention heads to focus on operands ({A} and {B}) and the operator, relaying this information to downstream MLPs. Over time, the MLPs progressively bolster {C} and diminish the effect of confused answers, carrying out the calculation to final results.

5.3. Precise SFT on Calculation-related Components.

Experimental details. We evaluate precise SFT on four mathematical datasets (GSM8K (Cobbe et al., 2021), AddSub (Hosseini et al., 2014), SingleEq (Koncel-Kedziorski et al., 2015), SVAMP (Patel et al., 2021)), and another two datasets (MMLU (Hendrycks et al., 2020) and CSQA (Saha et al., 2018)) to evaluate the generic ability. During training, we optimize the key components only and leave the other components unchanged. We gather all training data from four mathematical datasets, and perform SFT updating on top 32 key heads. Following (Yu et al., 2023), the gradient is rescaled by $\frac{H}{h}$, where H is the number of all heads in each layer, h is the number of updated heads in each layer. In practice, we train LLaMA2-7B and LLaMA2-13B with a learning rate of 2×10^{-5} and a batch size of 128 for 2 epochs. The warm up ratio and weight decay are set as 0.02 and 0.1 by default, respectively. All experiments are conducted on 8 NVIDIA A100 80GB GPUs.

Table 2: Ablative experiments on the number of tunable components. The default setting is shown in gray.

Precise SFT Setting	**Evaluation Metric**			
	Train Speed	Tuned Params.	GSM8K	MMLU
top-8 heads	58sam./sec.	0.017B	25.4	45.1
top-16 heads	52sam./sec.	0.033B	26.5	45.8
top-32 heads	50sam./sec.	0.067B	27.4	46.4
top-48 heads	46sam./sec.	0.101B	27.4	46.4
top-64 heads	40sam./sec.	0.134B	27.3	45.5
top-32 heads + top-3 MLPs	31sam./sec.	0.473B	28.0	45.2

Precise SFT improves mathematical ability. Supervised Fine-Tuning (SFT) is an effective approach for augmenting the mathematical capabilities of models by fine-tuning all parameters within LLMs. We term this all-parameter fine-tuning as Full SFT for clarity, and adopt the same training settings as Precise SFT. Table 1 presents the results of Full SFT and Precise SFT on the LLaMA2-7B and LLaMA2-13B models. Precise SFT effectively bolsters their mathematical capabilities, yielding an averaged increase of 15% on four distinct mathematical datasets. It matches or even surpasses the improvements made by Full SFT. For example, Precise SFT outperforms Full SFT by 5.5% on the SVAMP dataset and 2.8% on GSM8K, underlining its superior ability to enhance the mathematical prowess of LLMs. Full SFT suffers from the trade-off between mathematical and general capabilities (about 5% drops on MMLU and CSQA), while Precise SFT effectively maintains the model's original performance. A further advantage of Precise SFT is the drastic

reduction in training time, attributed to the substantially fewer parameter adjustments required (less than 1%). It results in a time reduction of at least threefold on LLaMA2-7B and LLaMA2-13B. Overall, Precise SFT offers an effective direction for boosting mathematical abilities for LLMs.

Ablative studies. The key issue with Precise SFT lies in determining the quantity and specific set of components to adjust. To demonstrate this, we experimented with varying numbers of heads and MLPs, with the results laid out in Table 2. We discovered that fine-tuning 32 heads yields the best average improvement across different numbers of involved heads. We also compared experiments with the introduction of MLPs. We observed that as more MLPs are added, the mathematical capability improves by 2.1%, but the general performance will decrease by 1.5% (results in Appendix G). Overall, the top-3 MLPs yielded the best comprehensive results. However, even the introduction of a single MLP can reduce computational efficiency by 15%. How to more precisely fine-tune MLPs will be explored in our future work.

More discussions. The above results underscore the potential of employing interpretability tools to analyze the inner mechanism of LLMs and to enhance their specific capabilities. However, there are several areas that require deeper investigation: (i) Our primary experiments and discussions center around the LLaMA2 series. The results presented in Appendix C demonstrate the potential for generalization across different LLMs, such as Mistral-7B (Jiang et al., 2023). For more rigorous considerations, it's necessary to perform specific adaptations on a broader range of LLMs. (ii) This work mainly focuses on interpreting the fundamental ability of "arithmetic calculation", since it's universally shared across various levels of complexity for mathematical problems. The results in Appendix E reveal that solving the math word problems requires a synergy of multiple skills including "text comprehension" and "arithmetic calculation", which is aligned with the findings in recent research (Opedal et al., 2024). It's imperative for continued research to investigate more complex mathematical problems. (iii) The potential of generalizing to more complex mathematical tasks like exponentiation (*e.g.*, "{A} to the power of {B} equals _") has been validated in Appendix D. An intriguing research direction would be to investigate the shared and distinct mechanisms across various mathematical tasks.

6. Conclusion

In this study, we have identified, analyzed, and fine-tuned the internal components responsible for the mathematical calculation capability of LLMs. The language models frequently involve sparse heads to particularly attend to operands and operators, and subsequent MLPs to work out answers. We apply the precise tuning on the calculation-related heads/MLPs for better mathematical capabilities, with less impact on non-mathematical tasks compared with tuning all parameters. These findings contribute to a better understanding of the inner mechanism of LLMs.

Acknowledgements

This work was supported in part by NSFC No. 62222117. YGZ and YMC were supported in part by NSFC/Research Grants Council (RGC) Joint Research Scheme under Grant: N_HKBU214/21; in part by RGC Senior Research Fellow Scheme under Grant: SRFS2324-2S02.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

Alayrac, J., Donahue, J., Luc, P., Miech, A., Barr, I., Hasson, Y., Lenc, K., Mensch, A., Millican, K., Reynolds, M., Ring, R., Rutherford, E., Cabi, S., Han, T., Gong, Z., Samangooei, S., Monteiro, M., Menick, J. L., Borgeaud, S., Brock, A., Nematzadeh, A., Sharifzadeh, S., Binkowski, M., Barreira, R., Vinyals, O., Zisserman, A., and Simonyan, K. Flamingo: a visual language model for few-shot learning. In *Advances in Neural Information Processing Systems*, 2022.

Barak, B., Edelman, B. L., Goel, S., Kakade, S., Malach, E., and Zhang, C. Hidden progress in deep learning: Sgd learns parities near the computational limit. *arXiv preprint arXiv:2207.08799*, 2022.

Bender, E. M., Gebru, T., McMillan-Major, A., and Shmitchell, S. On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of the 2021 ACM conference on fairness, accountability, and transparency*, pp. 610–623, 2021.

Bolukbasi, T., Pearce, A., Yuan, A., Coenen, A., Reif, E., Viégas, F. B., and Wattenberg, M. An interpretability illusion for BERT. *CoRR*, abs/2104.07143, 2021.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, 2020.